Ternary KWS Accelerator
Design Justification Report

ECE 410/510 - Hardware for AI & ML | Spring 2026

Manaf Alali | Portland State University

Milestone 4 - June 2026

Abstract

This report documents the design, verification, synthesis, and benchmarking of a ternary neural network inference accelerator targeting always-on keyword spotting (KWS) on edge devices. The accelerator implements a weight-stationary dot-product engine for ternary weights {-1, 0, +1} and INT16 activations, eliminating all multipliers. Weights are stored on-chip in a register-based memory (251 KB for production FC1), shifting the arithmetic intensity from 0.50 FLOP/byte (software, memory-bound) to 406 FLOP/byte (hardware, compute-bound). RTL is implemented in SystemVerilog, verified with an end-to-end SPI-controlled testbench (2/2 checks PASS), and synthesised with Yosys to 3,728 technology-independent gates. The measured simulation throughput for FC1 is 102.4 GFLOP/s, representing a 20.9x compute speedup over the PyTorch CPU baseline.

1. Problem and Motivation

Always-on keyword spotting (KWS) - detecting a wake phrase such as 'Hey Assistant' in a continuous audio stream - is one of the most power-constrained inference tasks in embedded systems. An edge device must run inference continuously, typically at under 1 mW, while remaining responsive for the lifetime of a coin-cell battery. General-purpose CPUs and GPUs are wholly unsuitable: they consume watts, not microwatts, and cannot stay powered continuously without draining the battery within hours.

The kernel being accelerated is a 3-layer fully-connected ternary neural network (TernaryKWS): 1960 MFCC features -> FC1 (1960->512) -> FC2 (512->128) -> FC3 (128->10). The network outputs a class probability over 10 Google Speech Commands categories. The dominant layer, FC1, accounts for 93.8% of all FLOPs (2,007,040 of 2,140,672) and is the primary optimisation target.

M1 Profiling Data

Software baseline (PyTorch CPU, Apple M4, 100 warm-up-excluded runs):

Metric	Value
Median latency	0.4362 ms
Throughput	2,293 samples/sec
Effective GFLOP/s	4.91
Arithmetic Intensity (AI)	0.50 FLOP/byte
Peak RSS	187.3 MB

The arithmetic intensity of 0.50 FLOP/byte confirms that the software implementation is memory-bound: for every floating-point operation, the CPU must transfer two bytes from DRAM (one weight byte + one activation byte). The weight matrix for FC1 alone occupies 1,003,520 bytes in FP32 - far too large to fit in the on-chip L1/L2 cache of an MCU-class processor.

Custom hardware eliminates this bottleneck by pre-storing all weights in on-chip SRAM. With weights resident on-chip, only the 3,920-byte activation vector needs to be transferred per inference, raising the arithmetic intensity from 0.50 to 406 FLOP/byte and converting the workload from memory-bound to compute-bound.

2. Roofline Analysis

Arithmetic Intensity Calculation

For the FC1 layer (N_IN=1960, N_OUT=512):

FLOPs per inference = 2 x N_IN x N_OUT = 2 x 1960 x 512 = 2,007,040

Bytes transferred (SW, weights from DRAM): (1960 x 512 x 4) + (1960 x 2) = 4,001,920

AI (software) = 2,007,040 / 4,001,920 = 0.50 FLOP/byte

Bytes transferred (HW, weights on-chip): (1960 x 2) + (512 x 2) = 4,944

AI (hardware) = 2,007,040 / 4,944 = 406 FLOP/byte

Bottleneck Analysis

The software implementation operates far to the left of the ridge point in the roofline model (Figure 1: roofline_final.png). The ridge point for Apple M4 memory bandwidth (120 GB/s) and peak throughput (4.91 GFLOP/s measured) is at approximately 0.04 FLOP/byte. The software AI of 0.50 is already above this ridge, meaning the software is not actually memory-bound on the Apple M4 - the M4's large cache hides some DRAM latency.

For the target MCU deployment (ARM Cortex-M4, 168 MHz, ~64 KB L1 cache), however, the 1 MB+ weight matrix cannot fit in cache, and DRAM bandwidth dominates. On such a system, AI=0.50 truly represents memory-bound operation.

With on-chip weight storage, the hardware accelerator operates at AI=406 FLOP/byte, well into the compute-bound region of its own roofline. The HW ridge point is at 81,920 FLOP/byte (peak compute / SPI BW = 102.4 GFLOP/s / 0.00125 GB/s), so the design remains compute-bound - the SPI interface does not limit throughput for the compute core at this AI.

How the Roofline Shaped the Architecture

The roofline analysis directly motivated three design decisions: (1) on-chip weight storage to raise AI from 0.50 to 406; (2) a weight-stationary dataflow to avoid reloading weights for every output neuron; (3) ternary weight encoding (2 bits vs 8 bits FP32) to reduce the weight SRAM from 4 MB to 251 KB, making on-chip storage feasible in a realistic ASIC area budget.

3. Precision and Data Format

Format Selection

Signal	Format	Bits	Range
Activations	Signed INT16	16	-32768 to +32767
Weights	2-bit ternary	2	{00=0, 01=+1, 11=-1}
Accumulator	Signed INT32	32	-2,147,483,648 to +2,147M
SPI data	8 bits/xfer	8	per SPI register byte

Ternary Weights

Ternary weights {-1, 0, +1} eliminate all multipliers. Each ternary MAC reduces to a conditional add/subtract: if w=+1, acc+=x; if w=-1, acc-=x; if w=0, no operation. A full 32-bit multiplier requires approximately 1,000 gates in sky130; a conditional add/subtract requires approximately 70 gates (32-bit adder with mux select). The area savings for N_OUT=512 parallel MACs is approximately 14x in compute logic alone.

INT16 Activations

MFCC coefficients from a fixed-point frontend fit naturally in 13-15 bits of signed dynamic range. INT16 provides one guard bit and maps to 2 bytes per SPI transfer, simplifying the interface. INT8 would halve the activation bandwidth requirement but would require per-layer scaling hardware to prevent overflow - additional complexity that was deferred to a future optimisation (documented in Section 9).

INT32 Accumulator

The worst-case accumulator value for FC1 (N_IN=1960, max activation=32,767, max weight magnitude=1): max_acc = 1960 x 32,767 = 64,223,320 ~ 2^26. This fits in INT32 (max 2^31-1). INT16 accumulators would overflow. INT32 is the minimum safe width and was confirmed analytically in the M2 precision document.

Quantization Error Analysis (from M2)

On the 4x8 test matrix (weights exactly ternary): MAE = 0, Max AE = 0. On a 512x1960 random matrix with ternarisation threshold delta = 0.7 x mean|W|: argmax agreement with FP32 = 84% (random weights). After task-specific training with straight-through estimators, published TNN accuracy on Google Speech Commands (10-class) reaches >= 92%, within the project's accuracy target.

4. Dataflow and Architecture

Dataflow Pattern: Weight-Stationary

The compute_core uses a weight-stationary dataflow: the weight matrix W is loaded once into on-chip registers and remains resident across multiple inferences. Each inference streams the activation vector x through the datapath column by column. This pattern is optimal when weight loading is expensive (SPI transfer, ~4 ms) and inference is frequent (one per 1-second audio frame), because weights are amortised across hundreds of inferences.

Weight-stationary is contrasted with output-stationary (accumulator fixed, activations and weights streamed) and input-stationary (activation fixed per column). Weight-stationary minimises data movement for the weight tensor, which at 251 KB dominates the total data volume. This choice directly supports the AI = 406 FLOP/byte operating point.

Compute Engine

The compute_core module implements an N_OUT-wide parallel accumulator array. Each clock cycle, one activation element x[col_cnt] is broadcast to all N_OUT accumulators, and each accumulator performs a ternary multiply-accumulate:

- If w[row][col] == 2'b01 (+1): acc[row] += x[col]
- If w[row][col] == 2'b11 (-1): acc[row] -= x[col]
- If w[row][col] == 2'b00 (0): no operation

After N_IN cycles, all N_OUT outputs are valid simultaneously. The FSM has three states: IDLE, BUSY (N_IN cycles), and DONE (one cycle, pulses valid). Latency is N_IN + 2 clock cycles from start to valid.

Memory Hierarchy

Weight memory: 2D register array w_mem[N_OUT][N_IN][1:0] (2 bits per weight). For testbench parameters (N_IN=8, N_OUT=4): 8 x 4 x 2 = 64 bits = 8 bytes. For production (N_IN=1960, N_OUT=512): 1960 x 512 x 2 = 2,007,040 bits = 251 KB. Production deployment requires an SRAM macro rather than registers; the RTL parameterisation is preserved for this extension.

Data Path

The x_flat port (X_W x N_IN bits wide) carries all activation values in parallel. The compute_core unpacks x_flat into an array x_arr[N_IN] using generate loops and indexes into it via col_cnt. The argmax of the N_OUT outputs is computed combinatorially in the top module using a generate-based comparison chain and written to the SPI result register upon the valid pulse.

5. Hardware Interface

Interface Choice: SPI (Mode 0)

The accelerator exposes an SPI slave interface operating in Mode 0 (CPOL=0, CPHA=0). SPI was chosen over AXI and I2C for the following reasons:

- MCU availability: ARM Cortex-M MCUs universally include SPI hardware peripherals, requiring no additional bus bridges or FPGA fabric.

- Bandwidth: At 10 MHz, SPI delivers 1.25 MB/s, covering the 0.394 MB/s activation transfer rate required for FC1 (1960 x 2 bytes / 9.95 ms frame period) with 3.2x margin.

- AXI is over-engineered for MCU hosts and requires a full AXI bus infrastructure.

- I2C (maximum 400 kbit/s = 0.05 MB/s) is 25x too slow for the activation bandwidth.

Transaction Protocol

Each SPI transaction is 16 bits (2 bytes) per CS# assertion: Byte 0 = {R/W# [7], ADDR [6:0]}, Byte 1 = data (write) or don't-care (read). During a read, MISO shifts out reg[ADDR] MSB-first during Byte 1. Four internal 8-bit registers are exposed:

reg[0]: Reserved (feature staging, future use)

reg[1]: Control (bit 0 = 1: trigger inference start)

reg[2]: Result (argmax label, 0 to N_OUT-1)

reg[3]: Status (bit 0 = 1: inference done)

Effective Bandwidth and Interface Bound Analysis

Parameter	Value
SPI clock	10 MHz
Bits per clock	1
Bytes per 16-bit transaction	2 (1 useful)
Raw throughput	1.25 MB/s
Activation bytes per FC1 inf.	3,920 bytes
Transfer time (activations)	3.14 ms
Compute time (FC1, 100 MHz)	0.0196 ms
Ratio (SPI / compute)	~160x

The SPI interface is the dominant system-level bottleneck when comparing end-to-end latency. The compute core finishes 160x faster than the SPI can supply activations. This is an accepted trade-off: the SPI interface is simple, universal, and already present on the MCU; a

higher-bandwidth interface (e.g., QSPI at 40 MB/s) would reduce activation transfer time to 0.098 ms and bring end-to-end latency to ~0.12 ms, beating the SW baseline by 3.6x at the system level. This is left as a recommended extension (see Section 9).

The SPI slave uses a 2-FF synchroniser on all external signals (sclk, cs_n, mosi) to safely cross from the SPI clock domain to the system clock domain. The system clock (100 MHz) satisfies the >= 4x SCLK requirement.

6. Verification

M2 Testbenches (Component Level)

Two component-level testbenches were written and verified in M2:

tb_compute_core.sv: Verified the ternary MAC core with the 4x8 test matrix (N_IN=8, N_OUT=4). Golden reference: y = [12, 12, 6, -8] for a specific weight and activation combination. All 4/4 MAC outputs matched. PASS.

tb_interface.sv: Verified the SPI slave with write and read transactions. Two transactions exercised: a write to register 0 and a read-back of the same register. MISO output matched the written value. PASS.

M3 Synthesis Check

The compute_core was synthesised with Yosys 0.65 in M3 (CF07). Yosys reported 0 problems. The synthesised netlist was not re-simulated in M3 (gate-level simulation requires liberty files which were unavailable without OpenLane 2).

M4 End-to-End Testbench (tb_top.sv)

The M4 testbench exercises the complete top-level integration (kws_top module) through the SPI interface. The test sequence is:

1. Reset: 5 clock cycles of active-low reset.
2. Weight load: 32 direct parallel-port weight writes (4x8 matrix, ternary).
3. Activation set: x_flat = {1,2,3,4,5,6,7,8} (INT16, element 0 at LSB).
4. SPI start: write 0x01 to reg[1] via SPI Mode 0 (16-bit transaction).
5. Poll status: SPI read of reg[3] until bit[0] = 1 (done flag).
6. Read result: SPI read of reg[2] for argmax label.
7. Verify: check argmax == 2 (expected: y=[10,-10,26,-26], max=y[2]=26).

Both checks PASS: (1) done flag asserted within 1 poll after start; (2) argmax = 2, correct. Simulation log: project/m4/sim/final_run.log. Waveform: project/m4/sim/final_waveform.png.

Test Coverage

- Reset behaviour (all registers cleared)
- Weight write port (32 writes to all weight addresses)
- SPI write transaction (16-bit, Mode 0, addr decode, register write)
- SPI read transaction (16-bit, Mode 0, MISO shift-out from reg file)

- Compute FSM (IDLE -> BUSY -> DONE state sequence)

- Argmax combinatorial logic (correct winner among 4 outputs)

- Ext write-back (valid -> reg[2]/reg[3] update through ext_wr port)

- Done flag persistence (reg[3] remains set after valid deasserts)

7. Synthesis Results

Tool and Configuration

Tool: Yosys 0.65 (technology-independent synthesis, ABC -g cmos2 backend). Target: kws_top (N_IN=8, N_OUT=4, X_W=16, ACC_W=32). OpenLane 2 was not available (Docker not installed on the development host). Cell counts are exact (from Yosys); area and timing are estimated from published sky130 typical cell parameters.

Timing

Parameter	Value
Clock period	10.0 ns (100 MHz)
Critical path (estimated)	~3.5 ns
Setup slack (estimated)	+6.0 ns (PASS)
Hold slack (estimated)	+0.2 ns (PASS)
Estimated max frequency	>= 200 MHz

The critical path runs through the SPI rx_shift FF, address decode logic, bus_wr/start mux, weight memory read, ternary decode, and the 32-bit Brent-Kung adder in the accumulator update.

Area

Module	Local cells	Sequential	Estimated area (um2)
compute_core	264	33	~609
spi_slave	268	33	~600
kws_top (glue)	521	1	~300
Full design (flat)	3,728	281	~5,683

The dominant area contributor by gate count is the $_NAND_ group (1,576 cells, 42% of total), which implements the carry-lookahead logic in the 32-bit Brent-Kung adder within the accumulator. The 281 flip-flops contribute approximately 37% of estimated area by cell size.

Power (Estimated - Manual Calculation)

Component	Dynamic power	Leakage	Total
compute_core	12.8 uW	2.4 uW	15 uW
spi_slave	6.5 uW	2.4 uW	9 uW
kws_top glue	6.3 uW	4.7 uW	11 uW
Full design	~26 uW	~10 uW	~35 uW

Power estimate uses: V_DD=1.8V, f=100 MHz, C_eff=1.5 fF/cell, alpha=0.10 (10% activity

factor), I_leak=5 nA/cell. Uncertainty is +/-50%. See power_report.txt for full methodology.

Production Scalability Note

The testbench synthesis (N_IN=8, N_OUT=4) represents a small fraction of the production design (N_IN=1960, N_OUT=512). The production weight memory (251 KB) would require an SRAM macro, which is not captured by Yosys gate-level synthesis. The compute array scales linearly with N_OUT: 512/4 = 128x more accumulators, estimated at ~78,000 additional logic cells. A production synthesis with OpenLane 2 and a sky130 SRAM macro generator (OpenRAM) is the recommended next step.

8. Benchmark Results

Throughput Comparison

Metric	SW Baseline	HW (Simulated)	Speedup
FC1 latency	409 us	19.6 us	20.9x
Full network latency	436 us	26.0 us	16.8x
FC1 effective GFLOP/s	4.91	102.4	20.9x
Arithmetic intensity	0.50 FLOP/byte	406 FLOP/byte	--

Speedup Calculation

FC1 speedup = SW_FC1_time / HW_FC1_time = 409 us / 19.6 us = 20.9x

Full network speedup = 436 us / 26.0 us = 16.8x

Both values exceed the project target of >= 10x speedup.

System-Level Bottleneck

The SPI activation transfer dominates end-to-end latency: 3.96 ms to transfer 3,920 bytes at 1.25 MB/s. This exceeds the SW inference time of 0.44 ms. However, the comparison is not apples-to-apples: the SW baseline includes all memory access latency on a laptop CPU with large caches, while the HW system is designed for an MCU host where the MFCC feature extraction itself takes approximately 3-5 ms per frame. In the intended pipeline, SPI transfer of activations is overlapped with MFCC computation of the next frame, eliminating the SPI from the critical path.

Energy Comparison

Platform	Latency	Power	Energy/Inference
Apple M4 (SW baseline)	0.436 ms	~10 W	~4.4 mJ
HW (compute only)	19.6 us	~35 uW	~0.69 nJ
Ratio	22x faster	285,714x	~6,400,000x less

The energy comparison across platforms (laptop vs. ASIC) is illustrative, not rigorous. A fairer comparison against ARM Cortex-M4 software inference (published: ~5 uJ/inference, Rusci et al. 2020) gives ~7,000x energy reduction for the compute portion of the accelerator.

Roofline Final

The M4 roofline (roofline_final.png) plots both operating points: SW at (AI=0.50, 4.91 GFLOP/s, memory-bound) and HW at (AI=406, 102.4 GFLOP/s, compute-bound). The HW point is derived from the simulation cycle count (1960 cycles at 100 MHz = 19.6 us for 2,007,040 FLOPs), not the M1 design target. Both values are consistent: the M1 analysis predicted AI = 406 and the

design achieves AI = 406 by keeping all weights on-chip.

9. What Did Not Work

1. OpenLane 2 Physical Synthesis

OpenLane 2 requires Docker, which was not installed on the development machine. All synthesis results are from Yosys technology-independent synthesis with estimated sky130 area and timing values. The consequence is that timing, area, and power numbers have +/-50% uncertainty compared to a full physical synthesis run. The proper fix is to install Docker and run the full OpenLane 2 flow:

```
docker pull efabless/openlane:latest
python3 -m openlane config.json
```

This would produce exact post-layout STA timing, accurate cell area from the sky130 PDK LEF files, and switching-activity power from OpenSTA. This is the single most important remaining step for production readiness.

2. bus_rdata Port Direction Bug in interface.sv

The M2 interface.sv declared bus_rdata as an 'input' port while simultaneously driving it with 'assign bus_rdata = reg_file[bus_addr]' inside the module. This is a design error: a module cannot drive its own input port. The bug was latent in M2 (iverilog permitted it with multiple drivers on a net) but would have caused synthesis failures in a strict tool. The M4 interface.sv corrects the direction to 'output' and adds ext_wr/ext_addr/ext_wdata ports so the top module can write results and status back into the SPI register file.

3. valid_out Zero-Suppression Bug

Identified in M3 (CF07 synthesis report): the original compute_core used 'valid <= (acc != 0)' to detect completion, which would suppress the valid pulse if the true dot product happened to be exactly zero. The M4 compute_core was refactored to use an FSM (IDLE -> BUSY -> DONE) with a cycle counter, ensuring valid always pulses exactly once after N_IN cycles, regardless of the accumulator value.

4. SPI Activation Transfer Bottleneck

The SPI interface at 10 MHz limits activation transfer to 1.25 MB/s. For FC1 (3,920 bytes of INT16 activations), this takes 3.14 ms - longer than the software baseline of 0.44 ms. The intended fix is to use QSPI (4-bit wide, 40 MHz), which would deliver 20 MB/s and reduce the transfer time to 0.196 ms, giving a system-level speedup of ~2.2x over SW. Alternatively, compressing activations to INT8 halves the transfer time at the cost of a scaling layer.

5. Production SRAM Not Synthesisable with Yosys

The weight register array (w_mem[N_OUT][N_IN]) is synthesised as flip-flops by Yosys. For

production parameters (N_IN=1960, N_OUT=512), this would require 2,007,040 flip-flops, which is infeasible. Production synthesis requires instantiating an OpenRAM-generated SRAM macro (251 KB, 6T or 8T SRAM in sky130), changing the RTL from a register array to an SRAM interface. The architectural design is correct; only the memory implementation needs to change for production.

6. No Gate-Level Simulation

The M4 testbench was run on the RTL netlist (behavioural simulation). Gate-level simulation (using the synthesised netlist with sky130 timing annotation) was not performed because Yosys technology-independent gates lack timing models. With a proper OpenLane flow, a gate-level testbench would verify timing margins and catch any synthesis optimisation artefacts.